

NeoBank360 Sprint 4 Work Documentation

Sprint 4 Theme

Sprint 4 added the analytics and admin-control layer on top of the existing NeoBank360 banking system.

The main goal was not to create new banking transactions, but to read the data already produced by previous modules and convert it into meaningful insights for users and administrators.

Sprint 4 focused on:

- Financial Insights Engine
- User Analytics Dashboard
- Admin Dashboard Engine
- Pending Approval Management
- System Monitoring
- Advanced Admin Controls
- Audit and activity visibility

Important implementation rule:

No separate analytics tables were required for financial insights. Analytics are generated from existing tables such as users, accounts, transactions, budgets, bills, rewards, loan_applications, loan_accounts, and audit log data.

Day 32: Financial Insights Backend

Session 1: Backend Endpoint - GET /api/insights/{userId}

What Was Implemented

The backend financial insights endpoint was implemented to return a personal financial summary for the authenticated user.

Endpoint:

```
GET /api/insights/{userId}
```

This endpoint is handled by `InsightsController`.

The controller receives the `userId` from the URL and also identifies the currently logged-in user from the JWT security context through `SecurityUtils`.

The important security rule is:

The user can only view their own insights.

That means:

- User 7 can access `/api/insights/7`
- User 7 cannot access `/api/insights/8`
- Cross-user access returns HTTP 403

Backend Flow

```
Angular Insights Dashboard
  |
  v
InsightsService.getInsights(userId)
  |
  v
GET /api/insights/{userId}
  |
  v
InsightsController.getInsights()
  |
  v
SecurityUtils.getCurrentUser()
  |
  v
InsightsServiceImpl.getFinancialInsights(userId, requesterId)
  |
  v
InsightsRepository aggregation queries
  |
  v
FinancialInsightsDTO
  |
  v
Angular charts and cards
```

Why This Design Was Used

The user ID is not trusted only because it comes from the URL. A malicious user could change the path manually in the browser or API client.

So the backend compares:

- `userId` from path variable
- `requesterId` from JWT authenticated user

This makes the endpoint secure even if someone tampers with frontend code.

Controller Logic

The controller method does three things:

1. Reads the path variable `userId`

2. Reads the authenticated user from the security context
3. Delegates the business logic to the service

Example flow:

```
Request path userId = 7
JWT authenticated userId = 7
Allowed

Request path userId = 8
JWT authenticated userId = 7
Denied with 403
```

Service Logic

The service performs:

- Ownership validation
- Income aggregation
- Expense aggregation
- Savings calculation
- Last 6 months trend preparation
- DTO assembly

Savings is calculated as:

```
savings = totalIncome - totalExpense
```

Negative savings are valid.

Example:

```
totalIncome = 10000
totalExpense = 13500
savings = -3500
```

This is important because a banking system must show the real financial condition. It should not hide overspending by converting negative savings to zero.

Session 2: Backend Aggregation Logic

Total Income Query

InsightsRepository uses aggregation queries to calculate total income.

Business meaning:

Total income means the sum of all CREDIT transactions from active accounts of the user.

Rules:

- Only CREDIT transactions are included
- Only active accounts are included
- Inactive accounts are ignored
- Result is returned as `BigDecimal`

Conceptual SQL:

```
SELECT SUM(t.amount)
FROM transactions t
JOIN accounts a ON t.account_id = a.id
WHERE a.user_id = :userId
      AND a.is_active = true
      AND t.transaction_type = 'CREDIT';
```

Total Expense Query

Total expense means the sum of all DEBIT transactions from active accounts.

Conceptual SQL:

```
SELECT SUM(t.amount)
FROM transactions t
JOIN accounts a ON t.account_id = a.id
WHERE a.user_id = :userId
      AND a.is_active = true
      AND t.transaction_type = 'DEBIT';
```

Trend Summary Query

The trend summary returns income and expense grouped by month.

The backend groups transaction data by:

- Year
- Month
- Transaction type

The service then normalizes the result into exactly 6 months.

This is important because charts must remain stable even if some months have no transactions.

Example:

```
Jan 2026 -> income 5000, expense 2000
Feb 2026 -> income 0, expense 0
Mar 2026 -> income 10000, expense 4500
Apr 2026 -> income 0, expense 1200
May 2026 -> income 8000, expense 3000
Jun 2026 -> income 6000, expense 4000
```

Even if February has no transactions, it is still returned with zero values.

DTO Returned

The backend returns `FinancialInsightsDTO`.

It contains:

- `totalIncome`
- `totalExpense`
- `savings`
- `trendSummary`

Each trend entry contains:

- `monthLabel`
- `totalIncome`
- `totalExpense`

Performance Benefit

Instead of loading all transactions into Java and then calculating totals, Sprint 4 uses database aggregation.

This is faster because:

- The database is optimized for `SUM`, `COUNT`, and `GROUP BY`
- Less data is transferred from MySQL to Spring Boot
- The frontend receives only final analytics data

Day 33: User Analytics Dashboard

Session 1: Frontend Summary Cards

What Was Implemented

The user dashboard received an Insights section that displays personal financial analytics.

The main component is:

```
Neobank-frontend/src/app/pages/insights/dashboard/insights-dashboard.component.ts
```

This component loads financial data from the backend and displays it in analytics cards and charts.

Data Loading Flow

```
InsightsDashboardComponent.ngOnInit()
  |
  v
loadInsights()
  |
  v
AuthService / local user data gives userId
  |
  v
InsightsService.getInsights(userId)
  |
  v
GET /api/insights/{userId}
  |
  v
Component state updated
  |
  v
Cards and charts rendered
```

Summary Cards

The dashboard shows cards such as:

- Total Income
- Total Expense
- Savings
- Net Worth
- Account Balance
- Outstanding Loans
- Reward Points

Why Cards Were Used

Cards are used because users need quick financial visibility.

Instead of reading tables, the user can instantly understand:

- How much money came in
- How much money went out
- Whether they saved or overspent
- How much debt is still pending

- How many rewards they earned

Loading State

The dashboard shows a loading state while API calls are in progress.

This improves user experience because the page does not look broken while waiting for backend data.

Error State

If the API fails, the component stores an error message and displays a friendly error block instead of leaving blank charts.

This is important in production because APIs can fail due to:

- Expired JWT
- Backend unavailable
- Network error
- Authorization failure

Session 2: Trend Chart

Chart Implementation

The current implementation uses Chart.js directly for rendering analytics charts.

The Income vs Expense chart uses:

- Month labels from `trendSummary`
- Income dataset
- Expense dataset

Example dataset:

```
labels: ["Jan 2026", "Feb 2026", "Mar 2026", "Apr 2026", "May 2026", "Jun 2026"]
income: [5000, 0, 10000, 0, 8000, 6000]
expense: [2000, 0, 4500, 1200, 3000, 4000]
```

Chart Colors

Income is shown with a green accent.

Expense is shown with a red accent.

This matches real financial meaning:

- Green means money received
- Red means money spent

Theme-Aware Charts

The component listens to theme changes and refreshes chart options.

This is important because chart text, grid lines, legends, and tooltips must remain readable in both dark mode and light mode.

Business Value

The trend chart helps the user answer:

- Am I spending more than I earn?
- Which months had higher expenses?
- Is my savings improving?
- Do I need to reduce spending?

Day 34: Admin Dashboard Engine

Session 1: Backend Endpoint - GET /api/admin/dashboard

What Was Implemented

The admin dashboard API was implemented to provide platform-level banking metrics.

Endpoint:

```
GET /api/admin/dashboard
```

This endpoint is admin-only.

It is protected with:

```
@PreAuthorize("hasRole('ADMIN')")
```

Returned DTO

The backend returns `AdminDashboardDTO`.

Important fields:

- `totalUsers`
- `totalActiveUsers`
- `totalLoans`
- `pendingApprovals`
- `totalTransactions`
- `platformSavingsRate`

Backend Flow


```
Admin Dashboard Component
  |
  v
AdminDashboardService.getDashboard()
  |
  v
GET /api/admin/dashboard
  |
  v
AdminDashboardController.getDashboard()
  |
  v
AdminDashboardServiceImpl.getDashboard()
  |
  v
AdminDashboardRepository aggregation queries
  |
  v
AdminDashboardDTO
  |
  v
Admin KPI cards and charts
```

Platform Savings Rate

The platform savings rate is calculated as:

```
platformSavingsRate = (totalIncome - totalExpense) / totalIncome * 100
```

Example:

```
totalIncome = 500000
totalExpense = 350000

platformSavingsRate = (500000 - 350000) / 500000 * 100
platformSavingsRate = 30%
```

If total income is zero, the backend returns zero to avoid division by zero.

Why Platform Savings Rate Matters

This metric tells the bank how financially healthy the customer base is.

High savings rate means:

- Customers are retaining more money
- Lower financial stress

- More potential for investment products

Low or negative savings rate means:

- Customers are spending more
- Higher credit dependency
- Potential loan or overdraft risk

Session 2: Pending Approvals Detail

Endpoint

```
GET /api/admin/pending-approvals
```

This endpoint returns pending approval items for administrators.

Current Sprint 4 logic focuses mainly on pending loan applications.

DTO Returned

PendingApprovalDTO contains:

- id
- module
- applicantName
- productName
- requestedAmount
- applicationDate
- status

Ordering Rule

Pending approvals are ordered oldest first.

This is important in banking operations because older applications should be reviewed before newer ones.

Business Value

Without this endpoint, admins would need to open the full loan module and manually search pending requests.

With this endpoint:

- Pending work is visible immediately
 - Admin workload is easier to track
 - Loan applications are less likely to be forgotten
-

Day 35: Admin Dashboard Layout

Session 1: Frontend Admin Structure

What Was Implemented

The admin side includes dedicated admin dashboard components and protected admin routes.

The project uses Angular standalone architecture, so admin pages are integrated through standalone components and route configuration instead of a traditional large NgModule-based structure.

Important frontend pieces:

- Admin dashboard component
- Pending approvals component
- User management component
- System health component
- System logs component
- Admin guard
- Admin dashboard service

Admin Guard

The admin guard protects admin routes.

Its job is to check:

- Is the user logged in?
- Does the user have ADMIN role?

If the user is not admin, the route is blocked.

Typical flow:

```
User tries /admin
  |
  v
AdminGuard checks JWT/user role
  |
  +-- ADMIN -> allow route
  |
  +-- CUSTOMER -> redirect to user dashboard
```

Why Admin Routes Need a Guard

Backend security is the final protection, but frontend guards improve user experience.

They prevent customers from seeing admin pages or clicking admin-only links.

However, even if someone bypasses the frontend guard, backend `@PreAuthorize` still blocks the request.

Session 2: KPI Summary Cards

Admin Dashboard Component

The admin dashboard displays executive-level platform metrics.

Important metrics include:

- Total users
- Active users
- Total transactions
- Loan counts
- Pending approvals
- Platform savings rate
- Credit and debit volume
- Loan disbursement analytics
- Audit event count

Period Filters

The admin analytics dashboard supports period-based analytics such as:

- 7 days
- 30 days
- YTD

This allows admins to compare short-term and long-term platform performance.

Why KPI Cards Were Used

Admins need quick operational awareness.

KPI cards help the admin immediately see:

- How many users exist
- How active the platform is
- How many loans need approval
- Whether transactions are increasing
- Whether customers are saving or overspending

Auto / Manual Refresh Concept

The dashboard is designed around reloadable live metrics.

When the refresh action is triggered, the component calls the backend again and redraws charts without requiring a browser refresh.

This is important because banking dashboards must show live data after loan approvals, transactions, bill payments, or user status changes.

Day 36: System Monitoring and Pending Approvals UI

Session 1: Backend Pending Approvals Refinement

Module Filter

The pending approvals endpoint supports module filtering.

Example:

```
GET /api/admin/pending-approvals?module=LOAN
```

This makes the endpoint extensible.

Today it can show loan approvals.

Later it can also support:

- Account opening requests
- KYC approvals
- High-value transaction approvals
- Dispute approvals

System Health Endpoint

Endpoint:

```
GET /api/admin/system-health
```

This endpoint returns live monitoring information.

Current system health fields include:

- Database status
- Active session/user count
- Server uptime
- Application health

Database Health Check

The backend executes a simple database query:

```
SELECT 1
```

If the query succeeds, database status is **UP**.

If the query fails, database status is **DOWN**.

Why SELECT 1 Is Used

SELECT 1 is lightweight and fast.

It checks whether the database connection is alive without reading business data.

This makes it suitable for a monitoring endpoint.

Session 2: Frontend Pending Approvals Grid

Pending Approvals Component

The pending approvals page displays approval items in a table/grid.

Columns include:

- Type
- Applicant
- Product / Item
- Amount
- Applied Date
- Action

Review Button

The Action column includes a Review button.

The Review button takes the admin to the loan application review area.

Flow:

```
Admin opens Pending Approvals
  |
  v
Clicks Review on a pending loan
  |
  v
Navigates to Loan Applications / Loan Decisions
  |
  v
Selected loan details open for review
  |
  v
Admin approves or rejects
```

Empty State

If there are no pending approvals, the UI shows a friendly empty message.

This is better than showing a blank table because it clearly tells the admin that no action is required.

Business Impact

The pending approval grid saves admin time.

Instead of manually searching loan applications, the admin can directly review the oldest pending requests from one operational queue.

Day 37: Advanced Admin Controls

Session 1: Backend User Management and Activity APIs

User Management Goal

Advanced admin controls allow bank administrators to manage platform users and monitor activity.

Important operations:

- View users
- Activate or deactivate users
- View user activity
- Track admin actions through audit logging

User List

The admin user list returns user details but must never expose password hashes.

This is a critical security rule.

The frontend only needs safe fields such as:

- User ID
- Full name
- Email
- Role
- Status
- Registered date

User Status Update

The admin can activate or deactivate users.

Important validation:

An admin cannot deactivate their own account.

Why:

If an admin deactivates themselves, the platform may lose administrative access or create an operational lockout.

User Activity

The user activity endpoint returns recent user activity, especially transaction-related activity.

It helps admins understand:

- What the user recently did
- Whether suspicious activity exists
- How active the customer is

Audit Logging

Admin actions are logged.

Tracked values include:

- Acting admin ID
- Action name
- Target resource type
- Target resource ID
- Timestamp

Why Audit Logging Matters

Banking systems must be accountable.

If an admin changes a user status, approves a loan, or views sensitive monitoring data, the system should have a trace.

Audit logs help with:

- Compliance
- Security investigation
- Internal review
- Debugging production issues

AOP Audit Logging Design

The project includes an audit logging approach that centralizes request/action logging instead of manually writing log code in every controller method.

This is useful because:

- Controllers stay clean
- Logging is consistent
- Every important admin action can be tracked in one style
- Future endpoints can be audited with less repeated code

The audit logging service uses its own transaction boundary so audit records can be stored independently from the main business operation.

Session 2: Frontend User Management UI

User Management Component

The admin user management screen displays users in a management table.

Columns include:

- User ID
- Full Name
- Email
- Role
- Status
- Registered Date
- Actions

Status Badges

Users are visually marked as:

- ACTIVE -> green badge
- INACTIVE -> red badge

This allows admins to quickly identify blocked or disabled users.

Activate / Deactivate Action

The action button allows the admin to change user status.

Before changing status, the UI should ask for confirmation because deactivation affects user access.

System Health Panel

The system health panel shows:

- Database status
- Server uptime
- Active session/user count
- Application health

Green indicators mean healthy.

Red indicators mean attention is required.

Complete Sprint 4 Backend Architecture

Main Backend Packages Used

controller

Contains REST APIs:

- `InsightsController`
- `AdminDashboardController`
- Admin/user/audit related controllers

Controllers receive HTTP requests, validate security, and call services.

service

Contains interfaces such as:

- `InsightsService`
- `AdminDashboardService`
- `SystemAuditLogService`

Services define business operations.

service.impl

Contains implementation classes such as:

- `InsightsServiceImpl`
- `AdminDashboardServiceImpl`
- `SystemAuditLogServiceImpl`

These classes contain the actual business logic, calculations, validations, and repository calls.

repository

Contains Spring Data JPA repositories.

Sprint 4 repositories use:

- `SUM`
- `COUNT`
- `GROUP BY`
- DTO projections
- Native queries where needed for date grouping

dto

Contains response objects such as:

- `FinancialInsightsDTO`
- `TrendEntryDTO`
- `AdminDashboardDTO`
- `PendingApprovalDTO`

- `SystemHealthDTO`
- `UserActivityDTO`

DTOs prevent exposing database entities directly.

security

Contains JWT and role-based security utilities.

Sprint 4 depends on the existing JWT system to identify:

- Current user ID
- Current role
- Whether the request is authenticated

Complete Sprint 4 Frontend Architecture

Main Frontend Areas

User Insights

Location:

```
src/app/pages/insights/dashboard
```

Responsibilities:

- Fetch personal insights
- Display financial cards
- Render charts
- Refresh dashboard data
- React to theme changes

Admin Dashboard

Location:

```
src/app/pages/admin/dashboard
```

Responsibilities:

- Fetch admin KPIs
- Display transaction analytics
- Display loan analytics
- Show platform-level performance

Pending Approvals

Location:

```
src/app/pages/admin/pending-approvals
```

Responsibilities:

- Fetch pending approval queue
- Filter by module
- Navigate admin to review screen

User Management

Location:

```
src/app/pages/admin/user-management
```

Responsibilities:

- Display users
- Show status chips
- Trigger activate/deactivate actions

System Health

Location:

```
src/app/pages/admin/system-health
```

Responsibilities:

- Show database status
- Show uptime
- Show active session count
- Show application health

Services

Frontend services connect Angular to Spring Boot:

- `InsightsService`
- `AdminDashboardService`

They wrap `HttpClient` calls and keep API logic outside components.

How Backend Connects With Frontend

User Insights Example

```
User clicks Insights in sidebar
  |
  v
Angular loads InsightsDashboardComponent
  |
  v
Component reads logged-in user ID
  |
  v
InsightsService sends GET /api/insights/{userId}
  |
  v
JwtInterceptor attaches Authorization header
  |
  v
Spring Security validates JWT
  |
  v
InsightsController receives request
  |
  v
SecurityUtils gets authenticated user
  |
  v
InsightsServiceImpl checks ownership
  |
  v
Repository runs aggregation queries
  |
  v
DTO returned to Angular
  |
  v
Cards and charts update
```

Admin Pending Approval Example

```
Admin opens Pending Approvals
  |
  v
Angular AdminDashboardService calls /api/admin/pending-approvals
  |
  v
JWT token sent by interceptor
  |
```

```
      v
Spring Security checks ADMIN role
      |
      v
AdminDashboardController handles request
      |
      v
AdminDashboardServiceImpl loads pending loans
      |
      v
Repository returns PendingApprovalDTO projections
      |
      v
Angular table displays queue
      |
      v
Admin clicks Review
      |
      v
Loan decision screen opens selected application
```

Important Business Logic in Sprint 4

1. Ownership Validation

User analytics are owner-only.

This prevents one customer from seeing another customer's financial data.

2. Active Account Filtering

Insights ignore inactive accounts.

This keeps analytics aligned with currently usable banking accounts.

3. Negative Savings Support

Savings are not clamped to zero.

This gives honest overspending visibility.

4. Six-Month Trend Padding

The backend always returns exactly six months.

This keeps charts stable and prevents frontend layout bugs.

5. Admin-Only Dashboard

Admin analytics are protected using role-based security.

Customers cannot access platform-wide metrics.

6. Oldest-First Pending Approvals

Pending approvals are ordered by application date.

This supports fair review workflow.

7. Health Check Query

The backend uses a lightweight DB query to verify database connectivity.

8. Audit Logging

Admin and sensitive actions are tracked for accountability.

Performance Decisions

Database Aggregation Instead of Java Loops

Sprint 4 uses repository-level aggregation queries.

This is better than loading thousands of rows into Java because:

- MySQL performs aggregation efficiently
- Network payload is smaller
- API responses are faster
- Memory usage is lower

DTO Projections

Repositories return only required fields.

For example, pending approvals do not need the full user, loan product, and application object graph.

They only need:

- Applicant name
- Product name
- Amount
- Date
- Status

This reduces unnecessary data loading.

Read-Only Transactions

Analytics methods are read-only operations.

Using read-only transaction boundaries helps Hibernate avoid unnecessary dirty checking.

Business Impact

For Customers

Sprint 4 helps customers understand their money better.

They can see:

- Total income
- Total expense
- Savings
- Spending trend
- Budget vs actual
- Loan burden
- Rewards growth

This turns NeoBank360 from a transaction app into a personal finance assistant.

For Admins

Sprint 4 helps bank administrators manage the platform.

Admins can see:

- User growth
- Active users
- Total transactions
- Pending loan approvals
- Loan analytics
- Platform financial health
- System health
- Audit activity

This makes the application more enterprise-ready.

Interview Explanation

If asked to explain Sprint 4, answer like this:

Sprint 4 added the analytics and admin-control layer to NeoBank360. On the backend, I created secured read-only analytics APIs using Spring Boot, Spring Security, and Spring Data JPA aggregation queries. For user insights, the API calculates total income, total expense, savings, and a six-month trend from existing transaction data. I also added strict ownership validation so users can only access their own analytics.

For admin analytics, I added admin-only dashboard APIs that calculate total users, active users, total loans, pending approvals, total transactions, and platform savings rate. The platform savings rate is calculated using

total platform income and expense from active accounts. I also added pending approval and system health endpoints so admins can review loan applications and monitor database/application status.

On the frontend, I integrated these APIs into Angular standalone components. The user Insights dashboard displays KPI cards and Chart.js visualizations. The admin dashboard displays executive KPIs, charts, pending approvals, user management, and health monitoring. JWT interceptors attach the token automatically, and route guards prevent customers from opening admin screens.

The most important technical decision was to calculate analytics using optimized database aggregation queries instead of loading all records into memory. This improves performance and keeps the APIs lightweight.

Interview Questions to Practice

Backend Questions

1. How does `/api/insights/{userId}` prevent cross-user access?
2. Why did you calculate income and expense using repository aggregation queries?
3. Why should inactive accounts be excluded from analytics?
4. How do you handle missing months in a six-month trend chart?
5. Why are negative savings allowed?
6. How is `platformSavingsRate` calculated?
7. What happens if total platform income is zero?
8. Why is `@PreAuthorize("hasRole('ADMIN')")` used on admin APIs?
9. What is the purpose of `SystemHealthDTO`?
10. Why is `SELECT 1` used for database health checking?

Frontend Questions

1. How does the Insights dashboard load data?
2. Why did you use a frontend service instead of calling `HttpClient` directly from many places?
3. How does the JWT interceptor help Sprint 4 APIs?
4. How do charts update after API data is loaded?
5. How do you make charts readable in dark mode and light mode?
6. What happens when the Insights API fails?
7. How does the Review button in Pending Approvals help the admin workflow?
8. How are admin routes protected on the frontend?

Security Questions

1. Why is frontend route protection not enough by itself?
2. What is the difference between authentication and authorization?
3. How does JWT identify the current user?
4. How does the backend know whether a user is ADMIN?
5. Why should password hashes never be returned from admin user APIs?

Performance Questions

1. Why are aggregation queries faster than Java-side calculations?
 2. What is the advantage of DTO projections?
 3. Why are read-only transactions useful for analytics APIs?
 4. How would you optimize `/api/admin/dashboard` if data volume increased?
 5. Which fields would you index for analytics performance?
-

Best Short Presentation Answer

Sprint 4 transformed NeoBank360 into a data-driven banking platform. I implemented personal financial insights for customers and executive analytics for admins. The backend uses secure Spring Boot APIs, JWT ownership validation, admin role checks, and optimized JPA aggregation queries. The frontend uses Angular standalone components and Chart.js dashboards to show income, expense, savings, loan, reward, and platform analytics. I also added pending approval management, system health monitoring, user activity visibility, and audit logging so the project feels closer to a real enterprise internet banking system.